SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing
(Common to all Branches)

Mode of Exam
OFFLINE

DEPARTMENT OF COMPUTING TECHNOLOGIES
SRM Nagar, Kattankulathur – 603203, Chengalpattu District, Tamilnadu

Academic Year: 2023-24 (ODD)

RegNo

Test: CLAT3 **Date: 8.11.2023**
Course Code & Title: 21CSS201T / COA **Duration: 100 minutes**
Year & Sem: II & III **SET B** **Max. Marks: 50**

Course Articulation Matrix:

Course Learning Outcomes (CLO)	At the end of this course, learners will be able to:	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12
CO-4	Analyze concepts of parallelism and multi-core processors	H	-	-	-	-	-	-	-	-	-	-	-
CO-5	Classify the memory technologies, input-output systems and evaluate the performance of memory system	H	M	-	-	-	-	-	-	-	-	-	-

Part – A
Instructions: Answer all (10 x 1 = 10 Marks)

Q. No	Question	Marks	BL	CO	PO	PI Code
1.	The stages of 3 stage pipelining are ____. a. Decode, Fetch, Execute b. Execute, Fetch, Decode c. Fetch, Decode, Execute d. Address generation, Fetch, Execute.	1	1	4	1.3	1.3.1
2	R_{2out} , Y_{in} R_{3out}, Select Y, Add, Z_{in} Z_{out}, R_{1in} The above sequence of opertations represents **a.** R3=R1+R2 **b.** R1=R2+R3 **c.** R2=R1+R3 d. R3=R2-R1	1	2	4	1.3	1.3.1
3	The contention for the usage of hardware device is called a. Data hazard b. Instruction hazard c. Structural hazard d. Stall	1	1	4	1.3	1.3.1
4	The conditional branch instruction outcome can be predicted before the execution by using a logic called a. Branch prediction b. Delayed branch c. Structural hazard d. Static prediction	1	1	4	1.3	1.3.1
5	Which addressing mode is capable of working without fetch operation? a. Register indirect b. Direct c. Indexed d. Immediate	1	1	4	1.3	1.3.1
6	What is the primary challenge of parallel computing related to shared resources? a) Scalability b) Data synchronization	1	1	5	1	2.1.2

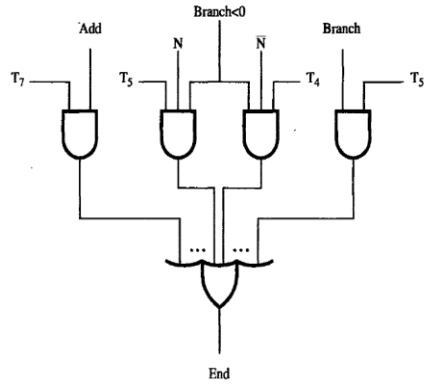
	c) Energy efficiency d) Algorithm complexity					
7	Which type of parallel system is typically used for tasks that involve repetitive operations on large data sets, like graphics rendering or scientific simulations? a) SISD b) SIMD c) MIMD d) MISD	1	1	5	1	2.1.2
8	In ARM assembly language, what instruction is commonly used for transferring data between memory and registers? a) ADD b) MOV c) SUB d) CMP	1	1	5	1	2.1.2
9	What is the purpose of the I/O operations in ARM architecture? a) To perform arithmetic calculations b) To manage memory allocation c) To interact with input and output devices d) To control processor temperature	1	1	5	1	2.1.2
10	What type of instruction encoding does ARM7 architecture use? a. Fixed-length instruction encoding b. Variable-length instruction encoding c. CISC (Complex Instruction Set Computer) encoding d. VLIW (Very Long Instruction Word) encoding	1	1	5	1	2.1.2

Part – B

Instructions: Answer any 4 (4 x 4 = 16 Marks)

11 Draw the generation of End control signal for single bus structure with proper expression.

- **END = T7.ADD+T5.BR+(T5.N+T4N̄).BRN+…..**

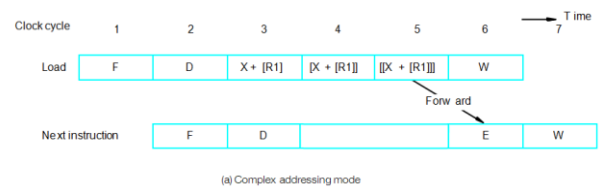


12 Illustrate the impact of complex addressing mode on instruction flow in the pipeline

- Addressing modes include simple ones and complex ones.
- In choosing the addressing modes to be implemented in a pipelined processor, we must consider the effect of each addressing mode on instruction flow in the pipeline:

  - Side effects
  - The extent to which complex addressing modes cause the pipeline to stall
  - Whether a given mode is likely to be used by compilers

Ex:



(a) Complex addressing mode

13 What is the sequence of operations for the instruction? Move R2,(R1)?

- $R1_{out}$,MAR_{in}
- $R2_{out}$,MDR_{in},Write
- MDR_{out}E, WMFC

14 Differentiate the Memory mapped I/O and I/O mapped I/O

	Memory Mapped Input/Output	Input/Output Mapped Input/Output
1.	Each port is treated as a memory location	Each port is treated as an independent unit
2.	CPU's memory address space is divided between memory and input/output ports.	Separate address spaces for memory and input/output ports.
3.	Single instruction can transfer data between memory and port.	Two instruction are necessary to transfer data between memory and port.
4.	Data transfer is by means of instruction like MOVE.	Each port can be accessed by means of IN or OUT instructions.

15 A CPU has 12 registers and uses 6 addressing modes. RAM is 64K x 32. What is the maximum size of the op-code field if the instruction has a register operand and a memory address operand?

Data:

number of registers = 12

addessing mode = 6

RAM size =64K × 32 = $2^{16} \times 2^5$

Formula:

Memory Capacity is of the form = $2^m \times 2^n$

Address lines required = m

Instrcution size = Data Lines = 2^n

number of bits = $\lceil \log_2 n \rceil$

number of register or number addressing modes

Calculation:

Instrcution size = Data Lines = 32

Address lines required = 16 bits

number of bits for a addressing mode = $\lceil \log_2 6 \rceil = 3$

number of bits for a register field = $\lceil \log_2 12 \rceil = 4$

Addressing Mode	op-code field	Register	Memory Address field
3 bits	x bits	4 bits	16 bits

3 + x + 4 + 16 = 32

∴ x = 9

op-code field = 9 bits

PART C

Instructions: Answer all (12 x 2 = 24 Marks)

16. A Write the sequence of control steps required for single bus structure for following instructions.

a) Add the immediate number NUM to register R1.

```
1: PCout, MARin, Read, Clear Y, Set carry-in to ALU, Add, Zin
 2: Zout, PCin, WMFC
 3: MDRout, IRin
 4: PCout, MARin, Read, Clear Y, Set carry-in to ALU, Add, Zin
```

```
 5: Zout, PCin
 6: R1out, Yin, WMFC
 7: MDRout, Add, Zin
 8: Zout, R1in, End
```

b) Add the contents of memory location NUM to register R1

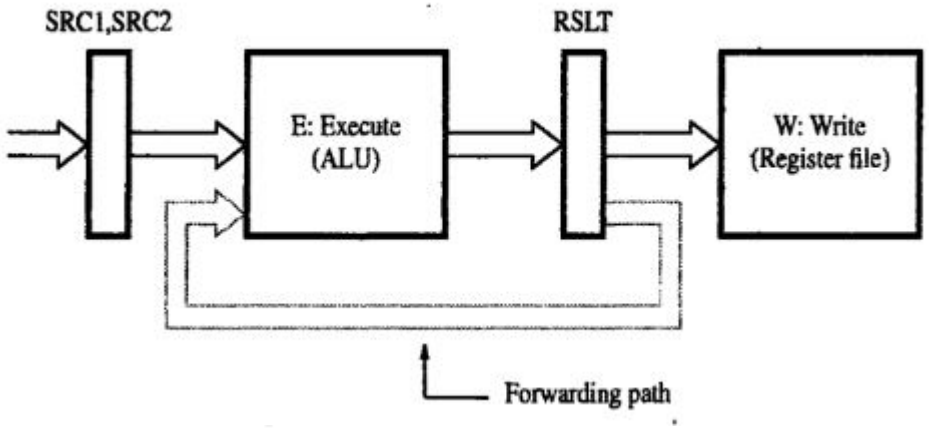
```
1: PCout, MARin, Read, Clear Y, Set carry-in to ALU, Add, Zin
 2: Zout, PCin, WMFC
 3: MDRout, IRin
 4: PCout, MARin, Read, Clear Y, Set carry-in to ALU, Add, Zin
 5: Zout, PCin
 6: R1out, Yin, WMFC
 7: MDRout, MARin, Read, WMFC
 8: MDRout, Add, Zin
 9: Zout, R1in, End
```

OR

16. B I1: MUL R2,R3,R4
I2: ADD R5,R2,R1

When the above instructions are executed parallel identify the type of hazard occurs? Specify the hardware and software solution for handling the hazard.

Data hazard occurs
h/w soln: operand forwarding



s/w soln:

I_1: Mul R2,R3,R4
NOP
NOP
I_2: Add R5,R4,R6

17 A. Imagine you are the lead architect at a tech company that specializes in developing real-time weather simulation software. The software needs to process vast amounts of data from various sources, simulate weather patterns, and provide real-time visualizations. To enhance the performance and reduce the time taken for simulations, you are considering implementing parallel processing techniques.

a. Identify the need for parallelism in this scenario. (2marks)

The primary need for parallelism in this scenario arises from the requirement to process vast amounts of data in real time. Weather simulations involve complex calculations and the integration of data from various sources. To achieve real-time visualizations and accurate predictions, the software must process this data swiftly. Parallelism can significantly reduce the time taken for these simulations by distributing the computational workload across multiple processors or cores.

b. Which type(s) of parallelism would be most suitable for this application and why? (2marks)

Data Parallelism: Given that weather simulations involve processing large datasets, data parallelism would be ideal. Different processors can handle different portions of the data

simultaneously. Task Parallelism: Different tasks, such as data collection, simulation, and visualization, can be executed concurrently, enhancing the software's efficiency.

c. **List two potential applications of parallelism in the weather simulation software. (2marks)**
Real-time Data Processing: Parallelism can be applied to process data from various sources like satellites, weather stations, and sensors concurrently, ensuring that the software has the most up-to-date information. Graphics Rendering: For visualizing weather patterns, parallel processing can be used to render graphics more swiftly, providing real-time visual feedback to users.

d. **Highlight two challenges you might face when implementing parallelism in this context. (2marks)**
Synchronization Issues: Ensuring that all parallel tasks or data processes are synchronized correctly is crucial. Any misalignment can lead to inaccurate simulations or visualizations. Data Dependencies: Some tasks might depend on the output of others. Managing these dependencies in a parallel environment can be challenging, as it's essential to ensure that all required data is available when needed

e. **Based on Flynn's classification, which architecture would be most appropriate for this software? Explain your choice. (4 marks)**
The most suitable architecture based on Flynn's classification would be Multiple Instruction, Multiple Data (MIMD). In MIMD, multiple processors operate independently on different data points and can execute different instructions. This flexibility is ideal for weather simulations where various tasks (like data processing, simulation, and rendering) can occur independently and simultaneously. – Explain with relevant diagrams.

OR

17 B Draw and explain the structure of ARM registers and processor modes

ARM Registers

- General-purpose registers hold either data or an address.
- Fig 2 shows the active registers available in user mode—a protected mode normally used when executing applications.
- The ARM processor has three registers assigned to a particular task or special function: *r13*, *r14*, and *r15*. They are frequently given different labels to differentiate them from the other registers.
- In Fig 2, the shaded registers identify the assigned special-purpose registers:
- **Register *r13*** is traditionally used as the **stack pointer (sp)** and stores the head of the stack in the current processor mode.
- **Register r14** is called the **link register (*lr*)** and is where the core puts the return address whenever it calls a subroutine.
- **Register *r15*** is the **program counter (pc)** and contains the address of the next instruction to be fetched by the processor.

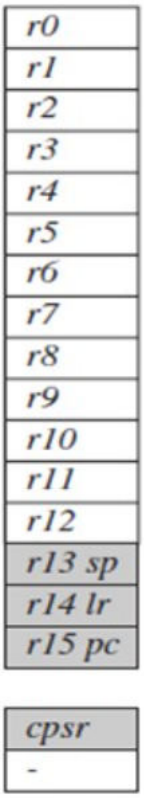


Fig 2: ARM Register in ***user mode.***

- The processor can operate in seven different modes. All the registers shown are 32 bits in size.
- There are up to **18 active registers**: **16 data registers** and **2 processor status registers**. The data registers are visible to the programmer as ***r0*** to ***r15***.
- Depending upon the context, registers r13 and r14 can also be used as general-purpose registers, which can be particularly useful since these registers are banked during a processor mode change.
- In ARM state the registers r0 to r13 are orthogonal—any instruction that you can apply to r0 you can equally well apply to any of the other registers. However, there are instructions that treat r14 and r15 in a special way.
- In addition to the 16 data registers, there are two program status registers: ***cpsr*** and ***spsr*** (the **current and saved program status registers**, respectively).

Processor Modes

- First 5 bits are signifies as mode selection
- The processor mode determines which registers are active and the access rights to the ***cpsr*** register itself.
- Each processor mode is either **privileged** or **nonprivileged:** A **privileged** mode allows full read-write access to the **cpsr**. Conversely, a **nonprivileged** mode only allows read access to the control field in the *cpsr* but still allows read-write access to the condition flags.
- There are seven processor modes in total: **six privileged modes** (**abort**, **fast interrupt request**, **interrupt request**, **supervisor**, **system**, and **undefined**) and one **nonprivileged** mode (**user**).

- ***Abort modes:***
  - The processor enters *abort* mode when there is **a failed attempt to access memory.**
- ***Fast interrupt request* and *interrupt request* modes:**
  - These two modes correspond to the two interrupt levels available on the ARM processor.
- ***Supervisor mode***
  - It is the mode that **the processor is in after reset** and is generally the mode that an operating system kernel operates in.
- ***System* mode**
  - It is a special version of user mode that allows **full read-write access** to the cpsr.
- ***Undefined* mode**
  - is used when the processor encounters an **instruction that is undefined** or not supported by the implementation.
- ***User mode***
  - is used for programs and applications.